

Production RAG

AICB-P2T3 · Ngày 18 · Từ Demo 60% Đến Production 85%+

Trần Quang Thiện
Ngô Thanh Tùng
VinUniversity · Phase 2 · Track 3 · Tuần 4 · 2026

HÃY SUY NGHĨ...

*“RAG pipeline demo chạy ngon, nhưng lên production accuracy chỉ **60%** — ingestion hay retrieval đang có vấn đề?*

Day 07–08 dạy RAG chạy được. Hôm nay: làm RAG đáng tin ở production.”



Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1 · Map & Compass
2 · Offline — ingest
3 · Online — retrieval
4 · Close the Loop
5 · The production reality check
6 · Enterprise-Scale Offline Ingestion & Data Lifecycle
7 · High-Performance Online Retrieval & Latency Mitigation
8 · Production Observability, Drift, & CI/CD

Hôm nay = **chất lượng ingestion + retrieval**. GraphRAG (Day 19), Multi-Agent (Day 20/23), RAGAS sâu & guardrails (Day 24), caching & reliability (Day 25), HITL (Day 27) học riêng.

Mục Tiêu Ngày 18

- **Chẩn đoán** pipeline hỏng — Error Tree, không đoán mò
- **OFFLINE**: hierarchical/late chunk, contextual, enrichment
- **ONLINE** retrieval: vượt hybrid+RRF — late interaction, learned sparse, fusion
- **ONLINE** rerank: chọn theo chất lượng / chi phí / ngôn ngữ
- **ONLINE** control flow: HyDE, Multi-Query, CRAG theo vấn đề
- **RAGAS** + diagnostic tree → sửa đúng tầng
- **Giới hạn** embedding — phần nào để dành ngày sau

Deliverable Cuối Ngày

`src/` (5 module hoàn chỉnh) + `analysis/` (RAGAS report + failure analysis)

- `m1_chunking.py` — hierarchical chunking (parent 2048 / child 256)
- `m2_search.py` — hybrid search (BM25 + Dense + RRF)
- `m3_rerank.py` — reranking top-20 → top-3
- `m4_eval.py` — RAGAS end-to-end (4 metrics)
- `m5_enrichment.py` — enrichment pipeline (summarize, HyQA, contextual prepend, metadata)
- `analysis/` — bottom-5 failure analysis map vào Error Tree

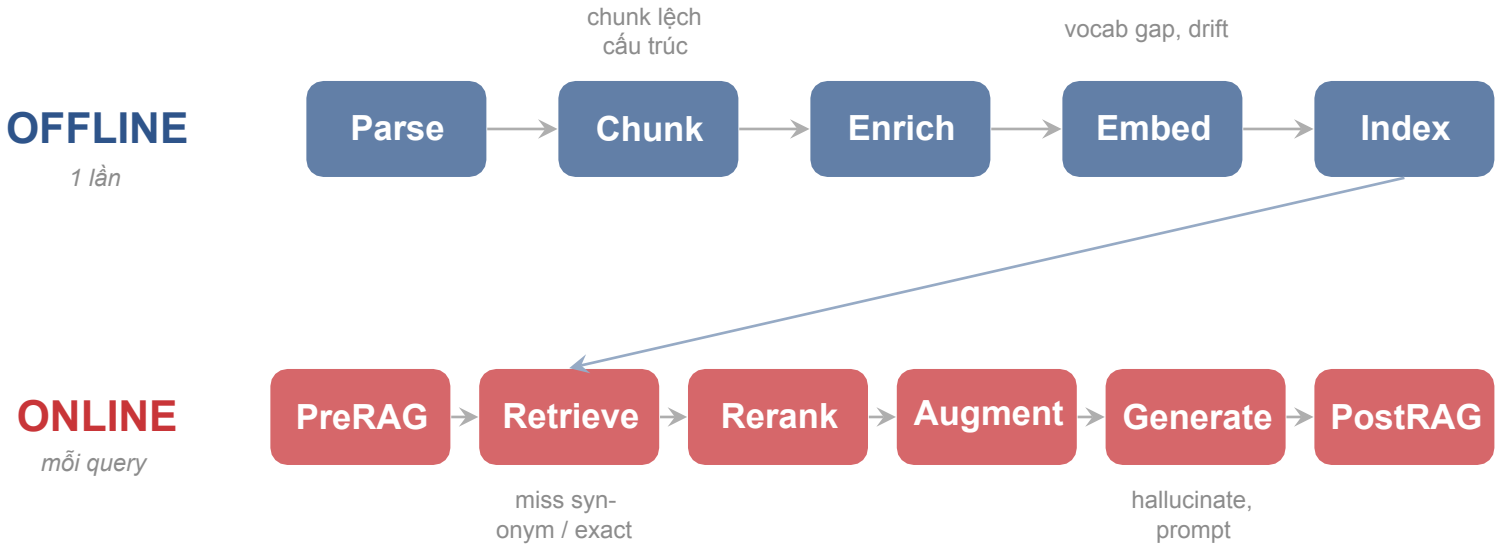
PHẦN 01

Map & Compass

1

Trước khi sửa: bản đồ pipeline (OFFLINE + ONLINE), la bàn Error Tree, và ca chạy TaiLieuBot xuyên suốt cả ngày

RAG = OFFLINE + ONLINE, Mỗi Bước Đều Có Thể Hỏng



Đọc Lại Bản Đồ: Mỗi Bạn Gọi Tên 1 Bước (1 phút/bạn)

VINUNIVERSITY

> Lần lượt mỗi sinh viên chọn **1 bước** trong pipeline và nói trong **1 phút**: bước đó *làm gì* và *có thể sai như thế nào*. Mục tiêu — cả lớp cùng nắm bản đồ trước khi đi sâu Phần 02–03.

OFFLINE (1 lần): Parse → Chunk → Enrich → Embed → Index

ONLINE (mỗi query): PreRAG → Retrieve → Rerank → Augment → Generate → PostRAG

Không bước nào "an toàn" — mỗi bước đều có thể là nơi pipeline hỏng.

Đánh Dấu: Mỗi Lỗi Thuộc Bước Nào? (5 phút)

VINUNIVERSITY

> Một AI assistant nội bộ trả lời câu hỏi chính sách. Bạn nhận được 4 case lỗi từ log production.

Nhiệm vụ (5 phút): Với mỗi case **đánh dấu bước** chịu trách nhiệm (**Retrieval / Rerank / Augment / Generate**) và đề xuất 1 fix:

1. User hỏi “nghỉ phép” nhưng tài liệu chỉ ghi “annual leave” → không tìm thấy gì.
2. Top-10 có chunk đúng ở vị trí #8, nhưng LLM chỉ đọc top-3 → trả lời thiếu.
3. Retrieve đúng chunk, nhưng câu trả lời thêm thông tin **không có** trong chunk.
4. Hai chunk mâu thuẫn (policy v2 vs v3), LLM chọn bản cũ.

> Mỗi case 1 dòng — `Case #` : bước → fix.
> Ví dụ `Case 1` : Retrieval → query rewrite / thêm synonym.

PHẦN 02

OFFLINE — Ingestion

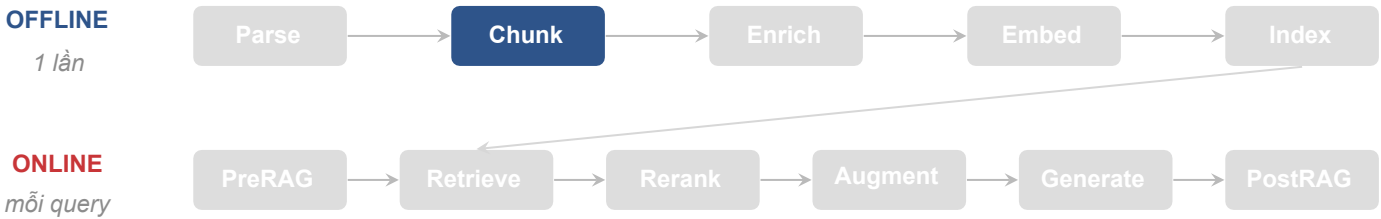
2

Nửa trái bản đồ: chunk → enrich → embed → index. Làm **một lần** lúc index, nhưng **quyết định trần chất lượng** cho mọi query về sau — retrieval không thể tốt hơn cái đã index

BƯỚC HIỆN TẠI

Chunking

Cắt tài liệu sao cho giữ ngữ cảnh



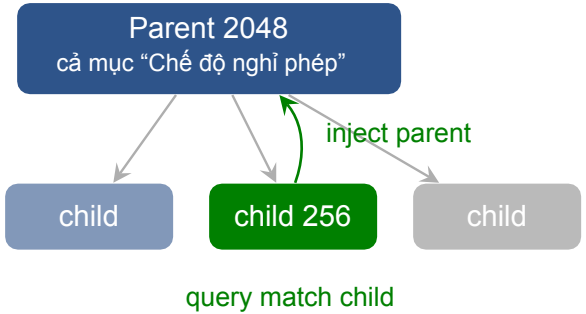
Chunking: Giữ Ngữ Cảnh Khi Cắt Nhỏ

Chiến lược	Tham số mặc định	Khi nào dùng
Fixed-size *Day 7*	512 tokens, overlap 64	Baseline, text đồng nhất
Semantic *Day 7*	cosine threshold 0.85	Đoạn đổi chủ đề, chat log
Hierarchical	parent 2048 / child 256	Default doc có cấu trúc
Structure-aware	theo heading/table	Markdown, HTML, PDF
Late chunking	embed doc trước, cắt sau	Tài liệu dài, tham chiếu ngược
RAPTOR	cây tóm tắt đệ quy	Hỏi tổng hợp xuyên đoạn

Hierarchical (default), **Late chunking**, **RAPTOR** (tổng hợp) dạy riêng ở 3 slide kế. Còn lại: tham khảo.

Hierarchical: Tìm Bằng Child Nhỏ, Trả Lời Bằng Parent Lớn

Mâu thuẫn Day 08: chunk *nhỏ* = retrieve chính xác nhưng **thiếu ngữ cảnh**; chunk *lớn* = đủ ngữ cảnh nhưng **nhiễu, khó match**.



Cách làm: index **2 tầng** — child 256 (để tìm) trỏ về parent 2048 (để đọc). Match trên child, nhưng **inject parent** vào LLM.

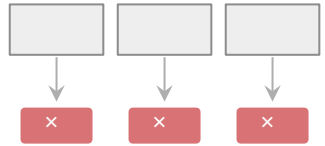
Lưu ý: **Default tốt** cho doc có cấu trúc (policy , manual). **Cách dùng:** LlamaIndex `AutoMergingRetriever` / LangChain `ParentDocumentRetriever` — khai báo parent+child size, retriever tự gộp. **Chi phí:** lưu cả 2 tầng.

Late Chunking: Embed Cả Document Trước, Cắt Sau

Cách thông thường: cắt chunk *trước* rồi embed từng cái rời → mỗi chunk **mù** phần còn lại của doc ("điều này", "công ty" — ai?).

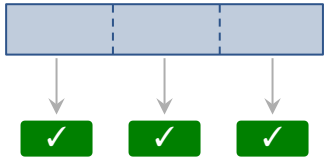
Late chunking: chạy token-embedding cho **cả document trước** (long-context encoder), *rồi mới* pool theo ranh giới chunk. Mỗi chunk vector **đã thấm** ngữ cảnh toàn cục.

Thường: cắt rời → embed riêng



vector *mù* ngữ cảnh

Late: embed cả doc → pool

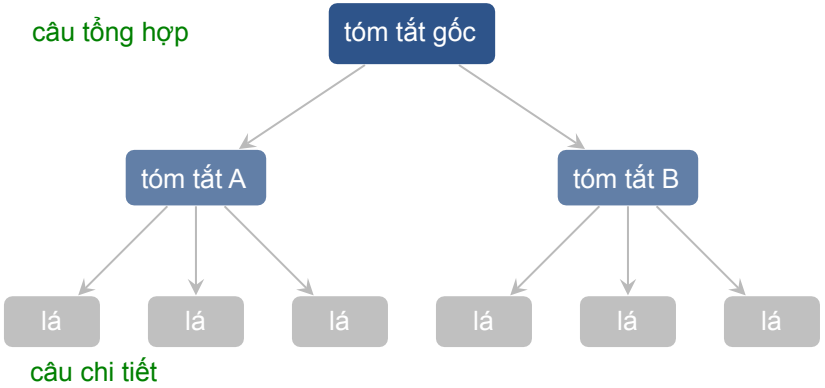


vector *thấm* ngữ cảnh

Lưu ý: **Khi nào:** tài liệu dài, nhiều tham chiếu ngược ("như đã nói ở trên"). **Cách dùng:** cần encoder long-context (Jina-v3, nomic) + thư viện hỗ trợ late chunking. **Khác contextual:** không gọi LLM/chunk — rẻ hơn, nhưng cần model hợp.

RAPTOR: Retrieval Có "Độ Cao" (Cây Tóm Tắt)

Cách thông thường: mọi chunk *cùng một mức* chi tiết. Câu hỏi tổng hợp ("toàn bộ chính sách phép đổi gì?") phải gom hàng chục lá → dễ miss, dễ tràn context.



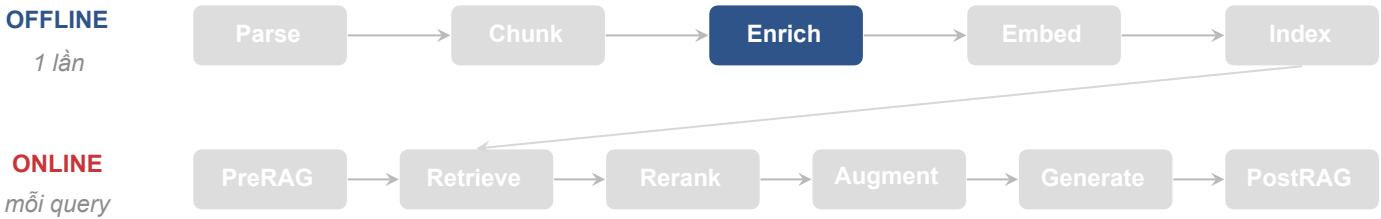
New approach: gom chunk → cụm → LLM tóm tắt, *lặp lên* thành **cây**. Retrieve ở **đúng độ cao**: chi tiết → lá; tổng hợp → nút cao.

Lưu ý: câu hỏi "theme / tổng hợp xuyên nhiều đoạn" mà chunk phẳng luôn trả thiếu.
Chi phí: build cây = nhiều LLM tóm tắt lúc index, index phình to. (Sarthi et al., ICLR 2024)

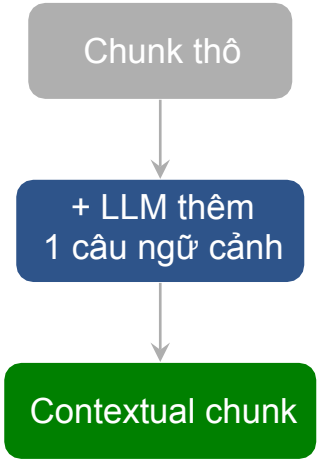
BƯỚC HIỆN TẠI

Enrich

Làm chunk giàu thông tin hơn trước khi embed



Contextual Embeddings (Anthropic, 2024)



"Đoạn này thuộc chính sách nghỉ phép 2026..." ghép vào trước chunk.

Vấn đề: chunk rời mất ngữ cảnh ("điều này" — điều gì?).

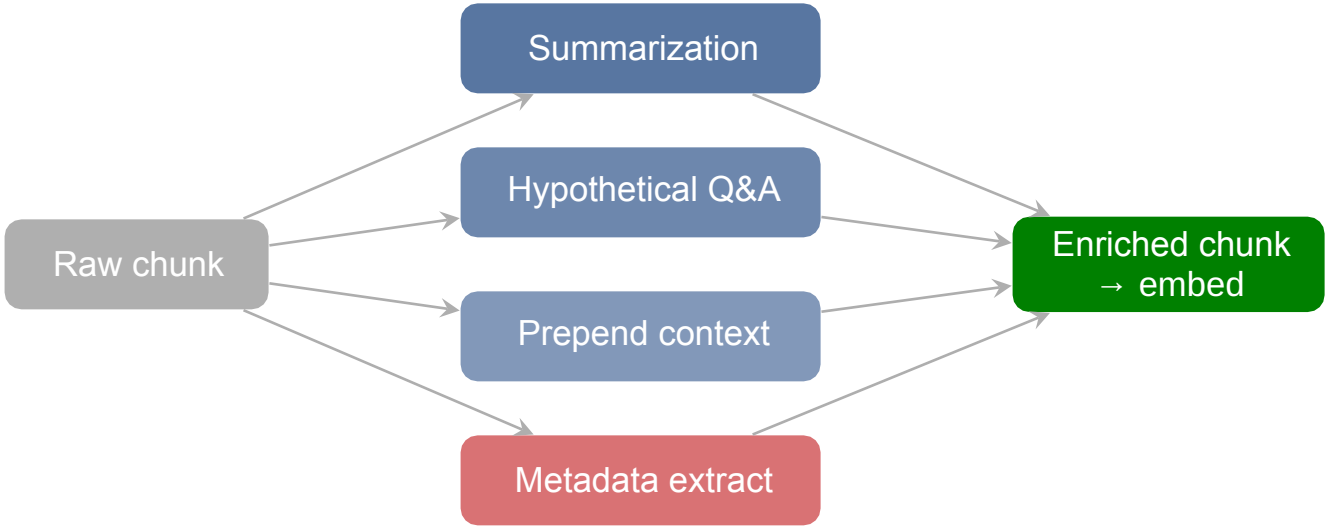
Hiệu quả (Anthropic) — *lỗi retrieval* giảm so baseline:

Cấu hình	Lỗi retrieval
Contextual (embed + BM25)	↓ 49%
+ reranking	↓ **67%**

Số liệu Anthropic (2024), tùy corpus. Chi phí: +1 LLM call/chunk lúc index (one-time, model rẻ). Nhồi cả document mỗi call → tốn token; dùng **prompt caching** để giảm. **Bổ sung**, không thay reranking.

Lưu ý: Với corpus văn bản dài / nhiều tham chiếu mơ hồ, đây thường là kỹ thuật OFFLINE **ROI cao** — trả tiền 1 lần lúc index. Đánh đổi: +1 LLM call/chunk lúc index (phải chạy lại mỗi lần re-index — đổi chunking, đổi model); corpus ngắn hoặc đã rõ

Enrichment Pipeline: Làm Chunk Dễ Tìm Hơn Trước Khi Embed



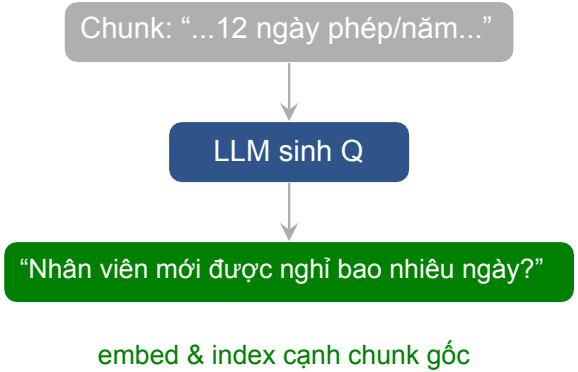
- **Summarization** — embed bản tóm tắt cạnh chunk gốc: lọc nhiễu, bám ý chính.
- **Hypothetical Q&A** — LLM sinh câu hỏi mà chunk trả lời được: nối vocab user ↔ tài liệu.
- **Metadata extract** — LLM rút entity, topic, ngày, nguồn → bật metadata filtering.

Tất cả là chi phí one-time lúc index. Dùng model rẻ (GPT-4o-mini, Haiku). Cân nhắc theo giá trị/query.

Hypothetical Q&A: Nối Vocab User ↔ Tài Liệu

Vấn đề: user hỏi bằng *câu hỏi*, tài liệu viết bằng *câu khẳng định*. Embedding của câu hỏi và đoạn văn thường **lệch** nhau.

Cách làm: lúc index, LLM sinh **các câu hỏi mà chunk này trả lời được**, embed chúng *cạnh* chunk. Query của user giờ match thẳng vào câu hỏi giả định.



Lưu ý: **Khi nào:** FAQ, hỏi-đáp chính sách — nơi user dùng từ khác tài liệu. **Cách dùng:** 1 LLM call/chunk lúc index (model rẻ), lưu Q-embeddings trỏ về chunk. **Họ hàng** HyDE (Phần ONLINE) — nhưng làm *offline*, không tốn call mỗi query .

Chọn Chunking Theo Loại Dữ Liệu — Không Có “One-Size”

VINUNIVERSITY

Chiến lược	Tốt cho (data)	Tránh khi	Tool chính	Tín hiệu RAGAS
Fixed-size	FAQ, doc ngắn đồng nhất	Doc cấu trúc chặt: cắt giữa ý (legal)	TokenTextSplitter	Context Recall thấp
Sentence Window	KB article, HR policy	Bảng / code: ranh giới câu vô nghĩa	SentenceWindow NodeParser	Faithfulness cao
Hierarchical	Spec, hợp đồng, ADR	Text phẳng: không có cây để khai thác	Hierarchical NodeParser	Context Precision cao
Semantic	Corpus hỗn hợp, research	Corpus lớn: embed từng câu, ingest chậm/đắt	SemanticSplitter NodeParser	Cân bằng mọi metric
Layout-aware (PDF)	PDF scan, report nhiều bảng	Doc text thuần: thừa, không có layout	PyMuPDF, pdfplumber	Vá recall ở bảng
Multimodal (slide/ảnh)	Deck, doc nhiều diagram	Corpus chỉ text: thừa chi phí vision	LlamaParse, GPT-4V	Context Recall (ảnh)

Bắt đầu từ **loại dữ liệu** của bạn → chọn cột “Tốt cho”. Nếu RAGAS báo metric ở cột cuối tụt → đang dùng sai chiến lược cho data đó.
Default an toàn: Hierarchical cho doc có cấu trúc.

PHẦN 03

ONLINE — Retrieval

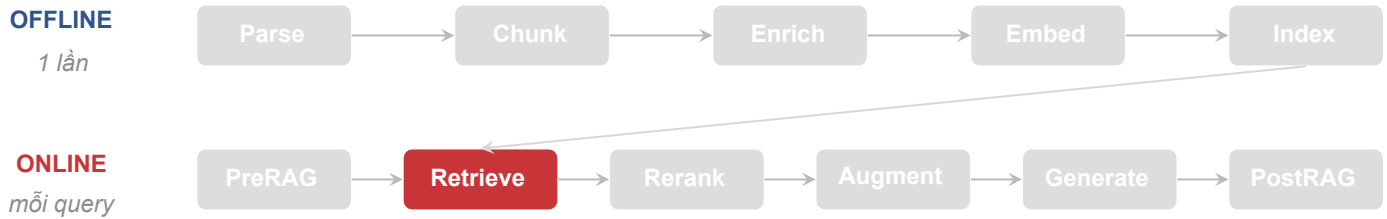
3

Nửa phải bản đồ: retrieve → rerank → control flow. Chạy lại **mỗi query**; mỗi vấn đề trong Error Tree có một fix riêng ở đây — không bật hết, bật đúng cái cần

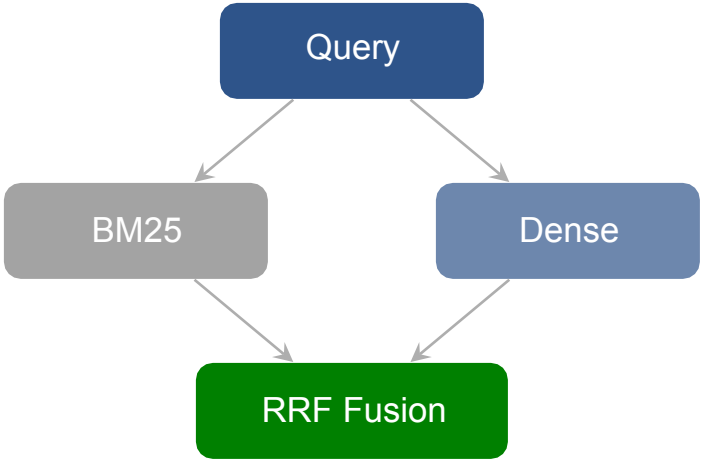
BƯỚC HIỆN TẠI

Retrieve

Tìm đúng chunk cho mỗi query



Hybrid + RRF: BM25 Bắt Exact, Dense Bắt Synonym (Nhắc Day 08)



	BM25	Dense	Hybrid
Exact keyword	☑	☐	☑
Synonym	☐	☑	☑
Latency	<5ms	~20ms	~25ms

Tiếng Việt: BM25 cần word segmentation (underthesea, VnCoreNLP) trước khi index.

$$\text{score}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$ k=60 (tinh chỉnh 10–100)

Không cần training, gộp rank từ nhiều nguồn khác thang điểm. Đây là baseline — phần sau là khi RRF **chưa đủ**.

Vấn Đề: Hybrid + RRF Vẫn Miss Chunk Đúng

Đoán — Câu trả lời nằm ở 1 token hiếm (mã “X7-204”). RRF gộp rank mức *document*. **Tại sao vẫn miss?** Nghĩ 30s trước khi xem bảng — gợi ý: cần so khớp **sâu hơn document** (mức token).

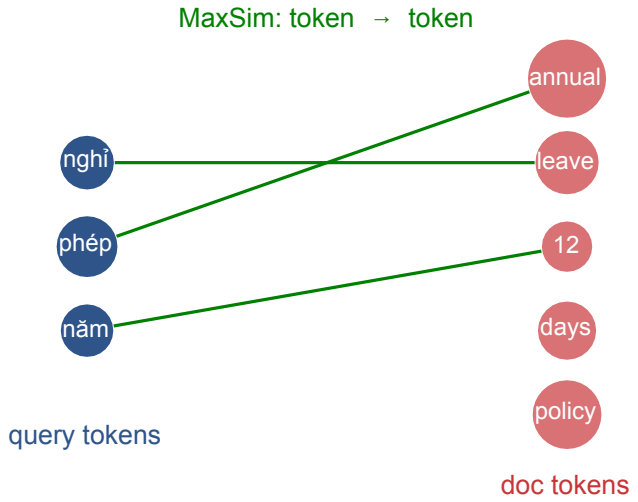
Kỹ thuật	Ý tưởng	Precision	Latency
RRF (baseline)	Gộp rank, không cần train	—	~25ms
Late interaction (ColBERT)	So khớp token-by-token (MaxSim); tốn storage multi-vector	+8–12%	~50ms
Learned sparse (SPLADE)	Sparse vector học được trên inverted index; storage gần BM25	+6–10%	~40ms
Tensor / weighted fusion	Outer product feature; cần label data	+10–15%	~80ms

*Con số tham khảo, tùy dataset/baseline; đổi precision lấy latency + storage — chọn theo SLA, không theo “mới nhất”. **3 slide sau**: từng kỹ thuật*

ColBERT: Thoát "Nút Cổ Chai 1 Vector" (Token-Level)

Cách thông thường: bi-encoder ép *cả đoạn* vào 1 vector → chi tiết token (tên riêng, số, thuật ngữ hiếm) bị nén mất. So cosine 2 vector.

New approach: giữ **1 vector / token**. Mỗi token query tìm token doc khớp nhất (**MaxSim**), cộng lại. Chi tiết được giữ; doc tính trước nên vẫn nhanh hơn cross-encoder.



Lưu ý: truy vấn cần *chi tiết exact* mà dense bỏ lỡ — không phải trả giá cross-encoder. **Cách dùng:** RAGatouille / Vespa / Qdrant multi-vector; cần index riêng (PLAID). **Chi phí:** nhiều vector/doc (storage lớn). (Khattab & Zaharia, SIGIR 2020)

SPLADE: BM25 “Học Được” — Sparse Nhưng Biết Mở Rộng

Cách thông thường: BM25 chỉ match *đúng từ xuất hiện*. “nghỉ phép” không khớp “annual leave” — vocab gap. Dense bắt synonym nhưng mờ token hiếm.

New approach: model **học** trọng số từ *và tự thêm từ liên quan* (term expansion) vào 1 sparse vector. Vẫn chạy trên **inverted index** (nhanh, rẻ như BM25) nhưng thông minh hơn.

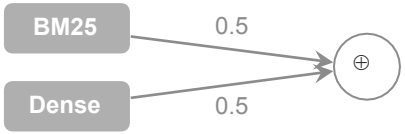
Chunk	“annual leave 12 days”
BM25	{annual, leave, 12, days}
SPLADE	{annual, leave, **vacation, holiday, pto**, 12, days}

*Term expansion (in xanh) đóng vocab gap **lúc index** — query “pto” vẫn tìm thấy.*

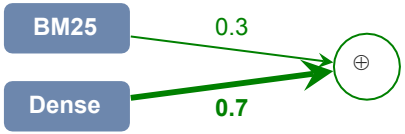
Lưu ý: vocab gap + cần tốc độ inverted index (không lưu multi-vector như ColBERT). **Cách dùng:** `naver/splade` encoder → Qdrant/Elasticsearch sparse vector. **Chi phí:** cần GPU lúc index để sinh sparse vector. (Formal et al., SIGIR 2021)

Tensor / Weighted Fusion: RRF Nhưng Có Học Trọng Số

RRF: cân bằng cứng 0.5 / 0.5



Fusion: học 0.3 / 0.7



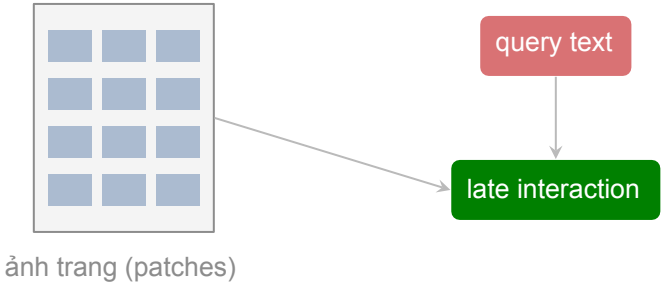
tin Dense hơn

"Label" là gì? — Cặp **(query → doc nào đúng)**. Nguồn: người gán nhãn relevance (vàng, đắt) *hoặc* click/feedback log (weak label, rẻ). Có label → train fusion; **không label → RRF**.

Lưu ý: Cách dùng: thu click-log trước, train fusion sau — đừng bắt đầu từ đây . **Mặc định vẫn RRF** tới khi đủ label. Đổi lấy 10–15% precision.

ColPali: Đừng Parse Tài Liệu — Hãy Nhìn Nó

Cách thông thường: PDF → OCR/parse → text → chunk → embed. Parsing *giòn*: bảng, layout, scan, biểu đồ mất sạch. "Garbage in".



New approach: vision-LLM embed *thẳng ảnh trang* (patch embeddings) + late interaction với query. **Bỏ qua cả pipeline parse** — bảng/biểu/layout giữ nguyên.

Lưu ý: tài liệu nặng layout/bảng/scan (tài chính, form, hợp đồng) — nơi parse *luôn* hỏng. **Cách dùng:** ColPali/ColQwen + Vespa/Qdrant multi-vector. **Chi phí:** lưu patch embeddings (lớn), cần vision model, tooling còn non. (Faysse et al., 2024)

Chọn Retriever Theo Nhu Cầu — Vượt Hybrid Khi Nào?

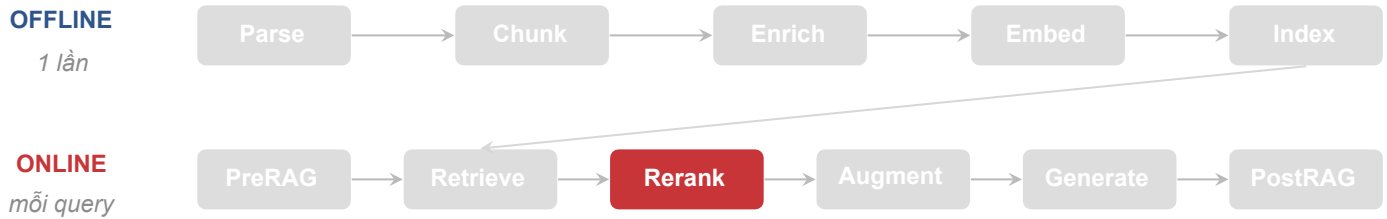
Kỹ thuật	Tốt cho (data)	Chi phí / Tránh khi	Tool chính	Tín hiệu RAGAS
Hybrid + RRF	Đa số ca: exact + synonym	Không học trọng số	Qdrant hybrid	Baseline (~ 25ms)
ColBERT *late interaction*	Cần precision token-level	Storage +8–12%	RAGatouille, Vespa	Context Precision cao
SPLADE *learned sparse*	Vocab mismatch, domain hẹp	Cần train/encode	SPLADE + inverted idx	Context Recall ↑
Weighted Fusion*tensor*	Có labeled data để tune	Cần label để học trọng số	Vespa rank profile	Precision ổn định ↑
ColPali *visual*	PDF/slide không parse được	GPU đắt; thừa nếu corpus text	ColPali, byaldi	Context Recall (ảnh)

Mặc định Hybrid + RRF, chỉ leo lên khi RAGAS chỉ rõ điểm đau: precision tụt → ColBERT; recall do vocab → SPLADE; có label → Fusion; ảnh/PDF → ColPali. **Đừng bật hết** — mỗi cái thêm latency/storage.

BƯỚC HIỆN TẠI

Rerank

Sắp xếp lại để chunk đúng lên top



Two-Stage Retrieval: Recall Rộng → Precision Hẹp



Retriever (bi-encoder) embed query & doc **riêng** → nhanh nhưng thô. Reranker (cross-encoder) đọc **cặp query+doc cùng lúc** → chính xác nhưng đắt. Lấy rộng rồi lọc tinh.

Lưu ý: ROI: tốn ~30–50ms/query , đổi lấy **+15–25% precision** (số tham khảo, tùy data). Khi latency cho phép, đây thường là một trong những tối ưu lời nhất.

TaiLieuBot: top-20 lẫn cả "nghỉ phép / nghỉ ốm / nghỉ thai sản" → reranker đẩy đúng mục "nghỉ phép" lên top-3.

Reranker Landscape: Chọn Theo Chất Lượng / Chi Phí / Ngôn Ngữ

Model	Chi phí	Khi nào dùng
Qwen3-Reranker	Open-weight	Top MTEB-rerank, đa ngôn ngữ, 0.6B–8B tùy budget
Cohere Rerank v3.5	API ($)	Lựa chọn managed phổ biến, đa ngôn ngữ
bge-reranker-v2-m3	Open-weight	Tiếng Việt tốt, 8K context, self-host
Jina Reranker v2	Open-weight	Đa ngôn ngữ, 8K context
ms-marco-MiniLM-L-12	Open-weight	Nhẹ, nhanh, thiên English
Flashrank	Open-weight	Siêu nhẹ, <5ms, edge
LLM-as-Reranker	$$$	GPT-4/Claude, chất lượng cao — chậm & đắt

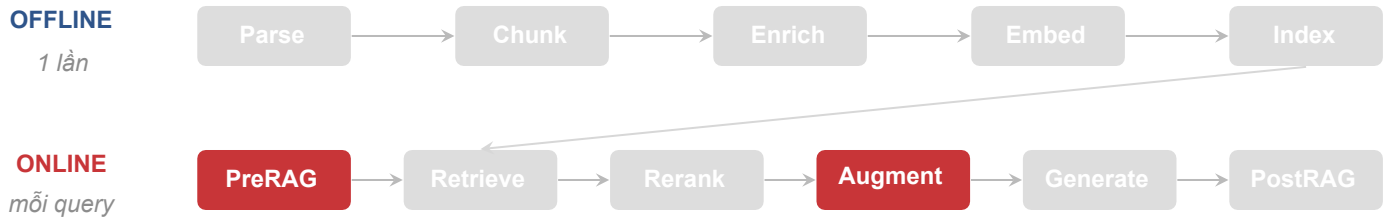
Open-weight = không phí license, nhưng vẫn tốn compute để self-host — không đáng kể nếu model nhỏ.

Chọn đi — TaiLieuBot: **tiếng Việt, self-host, latency cho phép**. Reranker nào? Chọn 1 trước khi xem gợi ý.

BƯỚC HIỆN TẠI

Control Flow

Định hình query & làm sạch context theo vấn đề



Mỗi Vấn Đề Một Pattern — Đừng Bật Hết

Vấn đề	Tầng	Pattern
Query lệch vocab so với corpus	PreRAG	HyDE
Câu hỏi nhiều phần / multi-hop	PreRAG	Multi-Query
Retrieve chất lượng thấp, cần fallback	PreRAG	Corrective RAG
Context dư thừa / mâu thuẫn	Augment	NLI filter, compression
Cần kiểm chứng nguồn	Augment	Citation injection

Lưu ý: Mỗi pattern sửa **một vấn đề** — các slide sau là chi tiết PreRAG / Augment (cache, reliability để dành Day 25).

Sửa Query Trước Khi Search — Theo Vấn Đề, Không Mặc Định

VINUNIVERSITY

HyDE *nhắc Day 8*

LLM sinh **câu trả lời giả định** → embed câu đó để tìm.

Nối vocab gap ("nghỉ phép" ↔ "annual leave"). Họ hàng Hypothetical Q&A (OFFLINE) — nhưng làm **mỗi query**.

Multi-Query *nhắc Day 8*

Tách query phức → nhiều **sub-query**, retrieve rồi gộp.

"Phép khác ốm thế nào?" → 2 sub-query rồi hợp kết quả.

Lưu ý: Đều **tốn +1 LLM call mỗi query** → bật theo vấn đề (HyDE: vocab gap; Multi-Query: nhiều phần), không mặc định. Corrective RAG (fallback khi retrieve kém) → slide kế.

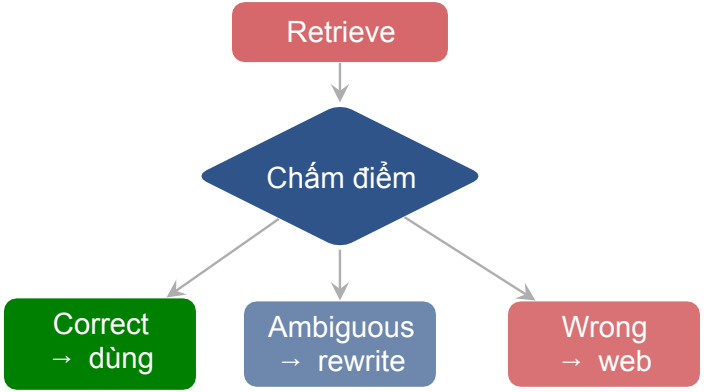
*Khác OFFLINE (trả 1 lần cho cả corpus): mỗi pattern ở đây = +1 LLM call **mỗi query**. TaiLieuBot: "nghỉ phép" bật HyDE; "phép khác ốm?" bật Multi-Query.*

Corrective RAG (CRAG): Tự Chấm Điểm Rồi Sửa Trước Khi Generate

VINUNIVERSITY

Cách thông thường: retrieve được gì thì nhồi thẳng vào LLM — kể cả khi chunk *lạc đề*. Rác vào → bịa ra.

CRAG: 1 evaluator **chấm điểm** chunk retrieve được. Theo điểm → rẽ nhánh: dùng luôn / viết lại query / **fallback web search**.



Lưu ý: **Khi nào:** corpus có lỗ hổng, câu hỏi ngoài phạm vi — thà fallback còn hơn bịa. **Cách dùng:** LangGraph node “grade → route”; evaluator = LLM rẻ hoặc classifier. **Chi phí:** +1 bước chấm điểm/query . Họ hàng Self-RAG (→ Day 23). (Y an et al., 2024)

Augmentation: Làm Sạch Context Theo Vấn Đề

Tầng giữa Retrieve và Generate — Top-k chunks hiếm khi sạch: dư thừa, mâu thuẫn, thiếu nguồn. Augment chèn bước **làm sạch** trước khi đưa vào LLM — bật **đúng bước** theo vấn đề (giống PreRAG), không bật cả 4.

Bước	Vấn đề	Cách làm	Tool
NLI verification	Chunk lạc đề / mâu thuẫn câu hỏi	Kiểm entailment query↔chunk, bỏ chunk "contradiction"	`nli-deberta-v3`
Compression	Prompt quá dài, nhiều nhiễu	Nén / bỏ câu ít liên quan, giữ ý chính	LLMLingua
Citation injection	Cần kiểm chứng nguồn	Gắn [1][2] mỗi câu, trace ngược về chunk gốc	prompt + map
Conflict resolution	2 chunk mâu thuẫn (v2 vs v3)	newest-wins (theo metadata) / LLM làm trọng tài	metadata + LLM

Lưu ý: Nhồi top-k thẳng vào prompt (như demo) **bỏ lỡ tầng này**. TaiLieuBot case #4: policy v2 vs v3 mâu thuẫn conflict resolution chọn v3 (newest-wins). →

PHẦN 04

Close the Loop & Limits

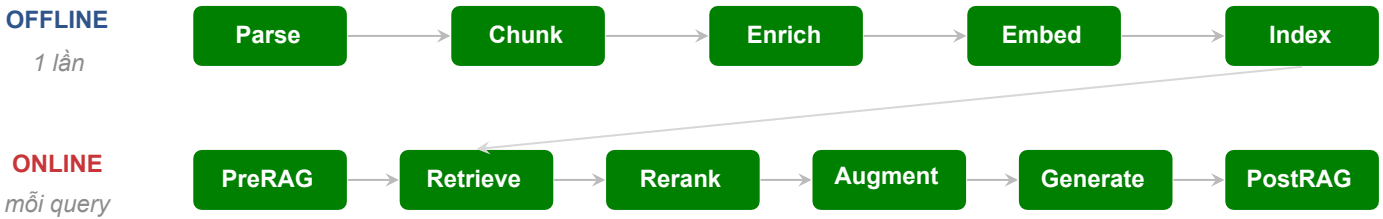
4

Đo bằng RAGAS → tra ngược về Error Tree (Phần I) để sửa đúng tầng — vòng lặp khép kín; rồi nhìn thẳng giới hạn embed-ding và ranh giới với các ngày khác

BƯỚC HIỆN TẠI

Close the Loop

Đo bằng RAGAS, tra Error Tree, sửa đúng tầng



Đo: RAGAS 4 Metric — 2 Trục, 2 Tầng

VINUNIVERSITY

RETRIEVAL — lấy đúng chưa?

Context Recall ≥ 0.75

Đã lấy *đủ* thông tin cần thiết?

Context Precision ≥ 0.75

Chunk liên quan có xếp *hạng cao* top-k?

Thấp → sửa tầng Retrieve/Rerank.

GENERATION — trả lời đúng chưa?

Faithfulness ≥ 0.85

Câu trả lời có *bám* context (chống bịa)?

Answer Relevancy ≥ 0.80

Trả lời đúng *trọng tâm* câu hỏi?

Thấp → sửa tầng Generate (prompt).

Một điểm tổng *che* chỗ hỏng. Tách retrieval vs generation → biết ngay **tầng nào** mất điểm để tra Error Tree. (RAGAS sâu, LLM-as-Judge, guardrails → Day 24.)

Metric Thấp → Sửa Gì (Error Tree Theo Metric)

Cùng bản đồ pipeline ở **Phần 01** — giờ vào từ cột **metric** thay vì vấn đề. RAGAS chỉ ra tầng yếu → tra về cùng la bàn → sửa → đo lại.

Metric thấp	Hành động (tầng nào trong bản đồ)
Context Recall	Hybrid (+BM25), HyDE, Multi-Query, metadata filter (Retrieval)
Context Precision	Reranking, tăng threshold, metadata filter (Rerank)
Faithfulness	Prompt chặt, hạ temperature, citation (Generate)
Answer Relevancy	Cải thiện prompt, mở rộng context (Generate)

(1) RAGAS baseline → (2) bottom-5 fail → (3) đọc metric thấp → tra bảng = bản đồ Phần I → (4) sửa → (5) so delta. **Thứ tự rẻ → đắt**: recall (hybrid) → precision (rerank) → faithfulness (prompt).

Giới Hạn Nền Tảng: 4 Điểm Mù Của Embedding

VINUNIVERSITY

Negation insensitivity

"được nghỉ" vs "**không** được nghỉ" → cosine cao nhưng nghĩa ngược.

Temporal blindness

Vector không mã hóa thời gian → chính sách **2024 vs 2026** cùng score.

Entity-swap

"phép **nhân viên**" vs "phép **quản lý**" → vector gần nhau, dễ nhầm.

Numerical blindness

"nghỉ **12** ngày" vs "nghỉ **20** ngày" → cosine cao, nhưng con số mới là đáp án.

Lưu ý: 4 ca này đều **cosine cao nhưng nghĩa khác** — embedding một mình không phân biệt được phủ định / thời gian / thực thể / con số. Không phải "kém", mà **chưa đủ**. Cách bù đắp → slide kế.

Giải Pháp Cho Lỗi Embedding Không Fix Được Bằng Data

- **Metadata filtering** (temporal, số, source) — sửa temporal & numerical blindness: lọc theo ngày / con số trước khi search.
- **NLI verification** post-retrieval — bắt negation/entity-swap: kiểm entailment query ↔ chunk.
- **Failure-aware routing** — phát hiện loại lỗi trước generate, đổi chiến lược (rewrite / decompose / từ chối).
- **GraphRAG** — câu hỏi quan hệ giữa nhiều entity: graph thay vì flat vector (→ **Day 19**).
- **Human-in-the-loop** — high-stakes: người duyệt trước khi trả (→ **Day 27**).

Không phải cứ thêm data hay đổi embedding model là fix được. **Chẩn đoán loại lỗi trước**, rồi chọn đúng công cụ.

Ranh Giới: Hôm Nay Dạy Gì, Phần Còn Lại Ở Ngày Nào

VINUNIVERSITY

> Chất lượng **ingestion (OFFLINE)** + **retrieval (ONLINE)**: chunking, contextual, enrichment, hybrid/rerank, query transform, augmentation — thường là phần "ăn điểm" lớn của RAG.

Chủ đề liên quan	Học sâu ở đâu
GraphRAG & Knowledge Graphs	**Day 19**
Multi-Agent / Agentic RAG patterns	**Day 20** (Multi-Agent), **Day 23** (Lang-Graph)
RAGAS sâu, LLM-as-Judge, Guardrails	**Day 24**
Caching, circuit breakers, reliability	**Day 25**
Human-in-the-loop UX	**Day 27**

Hôm nay chỉ dùng các chủ đề này ở mức "đủ để pipeline chạy & đo được"; chi tiết để dành đúng ngày.

PHẦN 05

The production reality check

5

Vượt qua giới hạn của Notebook để đối mặt với thực tế vận hành enterprise: Giải quyết các cạm bẫy về độ trễ, bùng nổ chi phí, bảo mật đa khách hàng và kiểm soát lỗi ngầm ở quy mô lớn.

The 90% Prototype Fallback: Tại sao Notebooks lại "nói dối"?

VINUNIVERSITY

Demo thành công ≠ Production ổn định

Mô hình chạy hoàn hảo trên vài chục tài liệu mẫu trong Jupyter Notebook, nhưng lập tức "vỡ trận" khi triển khai vào môi trường enterprise thực tế (hàng triệu tài liệu, cấu trúc phức tạp, truy cập đồng thời lớn).

1. Silent Failures (Lỗi ngầm)

Rủi ro: LLM tự tin bịa đặt

Hiện tượng: Model đưa ra câu trả lời sai hoặc bịa thông tin (hallucination) dựa trên ngữ cảnh thiếu/sai lệch, nhưng với giọng điệu cực kỳ thuyết phục.

Nguyên nhân Production:

- Text parser thất bại với PDF phức tạp, bảng biểu.
- Chunking cắt đứt ngữ cảnh quan trọng.
- Metric Context Recall < 0.75 gây thiếu dữ liệu.

2. Token Cost Explosions

Rủi ro: Bùng nổ chi phí

Hiện tượng: Chi phí API tăng phi mã ngoài tầm kiểm soát khi scale người dùng.

Nguyên nhân Production:

- Over-retrieval: Nhồi quá nhiều top-k context vào prompt để bù đắp cho Retrieval kém.
- Thiếu Reranking tối ưu kích thước cửa sổ ngữ cảnh.
- Lưu lượng truy cập đồng thời (concurrency) nhân bản số lượng Token Prompt mỗi giây.

3. Latency Degradation

Rủi ro: Độ trễ tăng vọt

Hiện tượng: Phản hồi chậm từ <1s (Local Notebook) nhảy lên 5–10s+ ở Enterprise.

Nguyên nhân Production:

- Scale dữ liệu từ 10 tài liệu lên 10M+ vectors gây nghẽn Vector Search.
- Chuỗi Pipeline RAG quá nhiều bước nối tiếp (PreRAG -> Retrieve -> Rerank -> LLM).
- Thiếu hạ tầng Caching và Streaming.

Giải pháp Kiến trúc: Cần tách bạch đo lường **Retrieval** (Context Recall/Precision) và **Generation** (Faithfulness/Relevancy) bằng RAGAS framework để khoanh vùng và khắc phục chính xác điểm nghẽn.

Four Pillars of Production RAG (Phần 1: Latency & Cost)

VINUNIVERSITY

Mục tiêu: Đạt chuẩn Enterprise RAG với Hiệu năng Tối ưu & Chi phí Bền vững

Cân bằng giữa cam kết thời gian phản hồi (SLAs) cực thấp và tối ưu hóa ngân sách vận hành khi scale hệ thống.

Trụ cột 1: Latency SLAs (Cam kết độ trễ)

Mục tiêu tối thượng: Pipeline toàn trình < 500ms

Phân rã ngân sách thời gian (Budget Allocation):

- **Network Hop:** ≈ 50ms
- **Vector Search:** ≈ 100ms
- **Reranking:** ≈ 50ms
- **LLM Time-to-First-Token (TTFT):** ≈ 250ms

→ Yêu cầu tối ưu hóa đa tầng từ mạng, hạ tầng Vector DB đến mô hình LLM.

Trụ cột 2: Cost Economics (Kinh tế học chi phí)

Bài toán cân đối đa chiều (Trade-offs):

- Chi phí lưu trữ Embedding vs Tài nguyên tính toán Vector Search.
- Độ trễ Cross-Encoder vs Token tiêu thụ của LLM.

Chiến lược tối ưu hóa:

- Cắt giảm các bước thừa thãi trong Pipeline.
- Tận dụng giải pháp Semantic Caching.
- Thu gọn Candidate Pool trước khi đẩy vào LLM Context.

Tóm lại: Đảm bảo **Latency < 500ms** và **Tối ưu Cost** là điều kiện tiên quyết để đưa RAG từ thử nghiệm (Prototype) ra vận hành thực tế (Production).

Four Pillars of Production RAG (Phần 2: Security & Freshness)

VINUNIVERSITY

Mục tiêu: Đảm bảo An toàn dữ liệu tuyệt đối & Đồng bộ hóa tức thì

Kiểm soát truy cập chính xác cấp độ dòng (Row-level) và duy trì tính thời sự của kiến thức doanh nghiệp mà không làm gián đoạn hệ thống.

Trụ cột 3: Security & Multi-Tenancy (Bảo mật & Đa khách hàng)

Thách thức cốt lõi:

- Ngăn chặn rò rỉ dữ liệu chéo giữa các phòng ban hoặc tenant.
- Tránh việc LLM truy cập hoặc tổng hợp các tài liệu mật không được phân quyền.

Giải pháp Kiến trúc:

- **Row-level metadata filtering:** Gắn nhãn ACL (Access Control List) trực tiếp vào từng Vector Chunk.
- **Search-time Enforcement:** Áp dụng bộ lọc phân quyền ngay **tại thời điểm tìm kiếm** (Search Time), triệt tiêu hoàn toàn rủi ro so với lọc hậu kỳ (Post-filtering).

→ Đảm bảo cách ly dữ liệu triệt để mà không làm tăng độ trễ pipeline.

Trụ cột 4: Data Drift & Freshness (Độ tươi & Lệch dữ liệu)

Thách thức cốt lõi:

- Doanh nghiệp cập nhật tài liệu liên tục (hợp đồng mới, chính sách sửa đổi).
- Tránh tình trạng LLM trả lời dựa trên thông tin lỗi thời (Data Drift).

Giải pháp Kiến trúc:

- **Real-time Ingestion:** Xử lý cập nhật (Updates) và xóa (Deletions) tài liệu theo thời gian thực.
- **Zero-Downtime Indexing:** Hỗ trợ Schema Evolution và cập nhật luồng dữ liệu mà **không cần rebuild lại toàn bộ Vector Index**.

→ Giữ cho tri thức hệ thống luôn chính xác, tươi mới và liên tục sẵn sàng.

Tóm lại: Bảo mật **Search-Time Metadata** và hạ tầng **Real-Time Data Freshness** là hai trụ cột bắt buộc để vận hành RAG an toàn, chuẩn xác trong môi trường Enterprise.

Production Architecture Blueprint: Luồng xử lý Request

VINUNIVERSITY

Mục tiêu: Tối ưu hóa toàn trình Pipeline RAG từ Tiếp nhận, Truy vấn đến Sinh phản hồi

Đảm bảo độ trễ thấp, tiết kiệm chi phí token và kiểm soát an toàn dữ liệu nghiêm ngặt cấp Enterprise.

1. Client Gateway

- Rate limiting
- Authentication
- SSL Termination
- Request Routing

→

2. API Layer

- Validation & Parse
- Tenant Context
- Session State
- Telemetry Tracing

→

3. Semantic Cache

- Vector Similarity Lookup
- Bypass LLM cho câu hỏi trùng
- Tối ưu chi phí & Latency

→

4. Hybrid Retrieval

- Dense Vector + BM25 Sparse Search
- Metadata ACL Filtering
- Distributed Vector DB

→

5. Accel. Reranker

- Cross-Encoder Scoring
- Tinh lọc Top-K Context
- Giảm nhiễu cho LLM

→

6. LLM Guard

- Prompt Injection Defense
- PII Anonymization
- Toxicity & Policy Filter

→

7. Generation

- Streaming LLM Response
- Citation Verification
- Output Guardrails

Tối ưu Chi phí: Semantic Caching

- **Lọc trùng thông minh:** Kiểm tra độ tương đồng ngữ nghĩa của truy vấn mới với các câu hỏi đã xử lý trước đó trong Cache.
- **Cắt giảm token:** Trả về kết quả ngay lập tức mà không cần gọi lại pipeline Retrieval hay LLM expensive generation.
- **Hiệu quả:** Giúp giảm tới 30-40% chi phí API LLM và đưa Latency phản hồi về < 50ms cho các truy vấn phổ biến.

Chất lượng & An toàn: Accelerated Reranker & LLM Guard

- **Accelerated Reranker:** Sử dụng Cross-Encoder tốc độ cao để đánh giá chính xác mức độ liên quan, nén context pool trước khi đưa vào LLM.
- **LLM Guard Layer:** Kiểm soát an toàn hai chiều (Inbound/Outbound) - chặn Prompt Injection, ẩn dữ liệu nhạy cảm (PII) và lọc nội dung độc hại.
- **Kết quả:** Đảm bảo câu trả lời luôn chuẩn xác, bám sát ngữ cảnh và tuân thủ chính sách an toàn Enterprise.

Tóm lại: Kiến trúc Production RAG chuẩn đòi hỏi sự phối hợp chặt chẽ giữa **Tối ưu Chi phí (Caching)** và **Kiểm soát An toàn (Guardrails)** để vận hành tin cậy ở quy mô lớn.

Architectural Breakdown: Quyết định thiết kế cốt lõi

VINUNIVERSITY

Mục tiêu: Đảm bảo tính mở rộng, độ tin cậy và an toàn tuyệt đối cho hệ thống Enterprise RAG

Phân tích chi tiết các thành phần hạ tầng quan trọng từ cổng tiếp nhận, lưu trữ vector quy mô lớn đến tầng kiểm soát an toàn.

1. Client Gateway & API Layer

- **Rate Limiting & Throttling:** Kiểm soát lưu lượng truy cập theo Tenant ID và API Key, ngăn chặn tấn công DDoS/abuse.
- **Enterprise Authentication:** Tích hợp SSO (OAuth2/OIDC, SAML) và cấp phát JWT token ngắn hạn.
- **Load Balancing & SSL:** Phân phối tải thông minh cho các request đồng thời lớn và mã hóa TLS/SSL end-to-end.
- **Context & Session Management:** Quản lý trạng thái phiên hội thoại và phân luồng request linh hoạt.

2. Distributed Vector DB Cluster

- **Scalability quy mô lớn:** Hỗ trợ Horizontal Scaling (Sharding & Partitions) đáp ứng hàng chục triệu bản ghi vector.
- **High Availability (HA):** Nhân bản dữ liệu (Replication Factor ≥ 3) đảm bảo không gián đoạn dịch vụ khi va đập node.
- **Tối ưu chỉ mục (Indexing):** Sử dụng thuật toán HNSW / IVF-PQ cho phép truy vấn chính xác cao với độ trễ thấp (<20ms).
- **Search-Time ACL Filtering:** Thực thi bộ lọc phân quyền Metadata cấp dòng (Row-level) trực tiếp khi tìm kiếm.

3. Guardrails Layer

- **Inbound Sanitization:** Chặn Prompt Injection, Jailbreak và phát hiện hành vi cố tình thao túng LLM.
- **Bảo mật dữ liệu (PII):** Tự động nhận diện và ẩn/mã hóa dữ liệu cá nhân nhạy cảm trước khi gửi tới LLM.
- **Outbound Verification:** Kiểm tra độc tính (Toxicity), tuân thủ chính sách nội dung và phát hiện Hallucination.
- **Safety Interception:** Ngăn chặn trả về thông tin vi phạm quy định Enterprise trước khi đến tay người dùng.

Tóm lại: Việc kết hợp chặt chẽ giữa **Client Gateway**, **Vector DB Cluster chuẩn Enterprise** và **Tầng Guardrails hai chiều** tạo nên hạ tầng RAG vững chắc, an toàn và sẵn sàng cho quy mô sản xuất lớn.

PHẦN 06

Enterprise-Scale Offline Ingestion & Data Lifecycle

6

Xây dựng pipeline dữ liệu chuẩn công nghiệp: Tối ưu hóa Ingestion với Change Data Capture (CDC), quản lý vòng đời index không gián đoạn (zero-downtime) và tiết kiệm tài nguyên tính toán qua MRL.

Thoát khỏi Batch Re-indexing: Giới thiệu CDC

VINUNIVERSITY

Vấn đề

Full Batch Re-indexing

- Tốn cực kỳ nhiều tài nguyên
- Chi phí tính toán lớn
- Gây gián đoạn hệ thống

Giải pháp

Change Data Capture

- Theo dõi thay đổi nguồn dữ liệu
- Cập nhật theo thời gian thực (real-time)
- Tối ưu cho dữ liệu doanh nghiệp

Cơ chế hoạt động

Xử lý sự thay đổi

- Chỉ xử lý tài liệu thêm mới, cập nhật, xóa
- Luôn đồng bộ dữ liệu
- Không cần rebuild từ đầu

Vận hành CDC: Upsert & Delete Triggers

VINUNIVERSITY

Nguồn dữ liệu tích hợp

- **Confluence:** Tài liệu quy trình & tri thức nội bộ.
- **SharePoint:** Khung lưu trữ & văn bản dùng chung.
- **SQL Databases:** Cơ sở dữ liệu quan hệ doanh nghiệp.

Upsert Trigger

- **Tự động định vị:** Phát hiện chính xác tài liệu vừa thay đổi.
- **Xử lý thông minh:** Cắt chunk & re-embed duy nhất phần dữ liệu mới.
- **Không gián đoạn:** Cập nhật Vector DB mà không ảnh hưởng tài liệu cũ.

Delete Trigger

- **Dọn dẹp lập tức:** Xóa sạch vector liên quan ngay khi gốc bị gỡ bỏ.
- **Ngăn ngừa ảo giác:** Triệt tiêu tình trạng hallucination do stale data.
- **Tối ưu bộ nhớ:** Giữ Vector DB luôn gọn nhẹ & chính xác.

Đảm bảo đồng bộ realtime giữa Data Sources và Vector DB — Giảm chi phí tính toán, loại bỏ hoàn toàn dữ liệu rác.

Zero-Downtime Index Swapping & Blue-Green Deployments

- **Thách thức:** Thay đổi embedding model hoặc chunking schema đòi hỏi re-index toàn bộ data mà không gây gián đoạn dịch vụ (zero downtime).
- **Blue-Green Strategy:** Duy trì Index hiện tại (Blue) để phục vụ live traffic, đồng thời khởi tạo Index mới (Green) ở môi trường ngầm.
- **Validation & Warm-up:** Thực hiện kiểm thử chất lượng, benchmark latency và warm-up cache trên Green Index trước khi cutover.
- **Atomic Switch:** Chuyển đổi traffic sang Green Index tức thì thông qua Aliasing / Router pointer, cho phép rollback nhanh nếu phát sinh lỗi.

Quản lý Alias trong Vector DB: Qdrant & Milvus

VINUNIVERSITY

Khái Niệm Alias

Lớp Trung Gian

- **Định nghĩa:** Là con trỏ trỏ trực tiếp đến collection thực tế trong Vector DB.
- **Cơ chế:** Giúp ứng dụng giao tiếp qua tên Alias cố định thay vì gọi trực tiếp tên Collection.
- **Mục đích:** Trừu tượng hóa tài nguyên, tách rời ứng dụng khỏi hạ tầng dữ liệu bên dưới.

Lợi Ích Vận Hành

Chuyển Đổi & Rollback

- **Chuyển đổi tức thời:** Đổi phiên bản index mượt mà chỉ bằng một câu lệnh trỏ lại Alias (Zero-Downtime).
- **Rollback lập tức:** Khôi phục về index cũ ngay tức thì nếu index mới gặp lỗi retrieval.
- **An toàn tuyệt đối:** Đảm bảo hệ thống luôn sẵn sàng phục vụ người dùng.

Công Cụ Hỗ Trợ

Vector DB Hiện Đại

- **Qdrant:** Hỗ trợ API Alias Management để swap collection cực nhanh.
- **Milvus:** Tích hợp sẵn tính năng Alias trỏ linh hoạt giữa các collection.
- **Tiêu chuẩn:** Là tính năng cốt lõi bắt buộc cho hạ tầng Production RAG/Search.

Tối ưu hóa tài nguyên với Matryoshka Representation Learning (MRL)

VINUNIVERSITY

Đặt vấn đề & Giải pháp MRL

- **Bài toán:** Vector đầy đủ (1536-d) tốn rất nhiều bộ nhớ RAM và băng thông tính toán.
- **Giải pháp MRL:** Vector nhúng được huấn luyện có cấu trúc lồng nhau (nested dimensions).

Chiến lược tìm kiếm 2 giai đoạn (Two-Stage Retrieval)

1. Coarse Filtering (Lọc thô)

Dùng vector nén kích thước nhỏ **(128-d)** để lọc thô siêu tốc trên toàn bộ cơ sở dữ liệu lớn.

→

2. Reranking (Xếp hạng lại)

Chỉ dùng vector đầy đủ **(1536-d)** để tính toán chính xác cho tập ứng viên rút gọn ở bước cuối.

Tối ưu vượt trội

Giảm **>90%** chi phí lưu trữ & độ trễ truy vấn nhưng vẫn giữ nguyên **98%+** độ chính xác so with việc tìm kiếm thô bằng vector 1536 chiều.

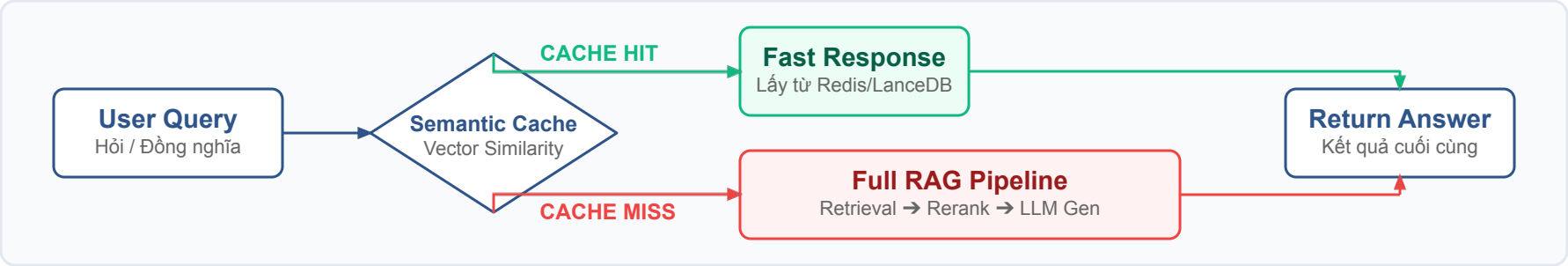
PHẦN 07

High-performance Online Retrieval & Latency Mitigation

7

Tăng tốc truy vấn trực tuyến dưới 500ms: Ứng dụng Semantic Caching thông minh, tinh chỉnh hybrid search động, tăng tốc phần cứng cho Reranker và lựa chọn kiến trúc Vector DB tối ưu.

Semantic Caching: Chặn đứng Request Trùng lặp



1. Vấn đề

Người dùng liên tục lặp lại câu hỏi giống nhau hoặc đồng nghĩa (VD: *"Chính sách bảo hành?" ≈ "Quy định đổi trả?"*).

2. Giải pháp Semantic Cache

Lưu trữ trực tiếp các cặp **[Query Embedding ➔ Generated Answer]** trong cơ sở dữ liệu như Redis hoặc LanceDB.

3. Tối ưu chi phí & Hiệu năng

Bỏ qua hoàn toàn Retrieval, Rerank và LLM Generation khi hit cache, tiết kiệm **40%–60% chi phí vận hành**.

4. Giảm độ trễ Phản hồi

Trả kết quả tức thì (Latency < 50ms) cho các truy vấn trùng lặp ngữ nghĩa thay vì chờ toàn bộ pipeline xử lý.

Vận hành Cache: Ngưỡng Cosine Similarity

VINUNIVERSITY

Cơ chế khớp nghĩa

Tính khoảng cách vector (Cosine Similarity) giữa câu hỏi mới và các key trong cache.

Ngưỡng Quá Cao (> 0.98)

- **Tỷ lệ Cache Hit thấp:** Bỏ sót nhiều câu hỏi tương đương về mặt ngữ nghĩa.
- **Lãng phí tài nguyên:** Bắt buộc system chạy lại toàn bộ RAG Pipeline không cần thiết.
- **Tăng độ trễ:** Người dùng phải chờ lâu hơn cho các truy vấn trùng lặp.

Ngưỡng Quá Thấp (< 0.85)

- **Hallucination do Cache:** Trả về câu trả lời cũ cho câu hỏi mang ý nghĩa khác.
- **Sai lệch thông tin:** Tự động khớp nhầm các intent nguy hiểm hoặc trái ngược.
- **Giảm độ chính xác:** Ảnh hưởng nghiêm trọng tới trải nghiệm người dùng.

Khuyến nghị Production

- **Mức ngưỡng tối ưu:** Duy trì cấu hình ổn định trong khoảng **0.92 – 0.95**.
- **Cơ chế TTL (Time-To-Live):** Đặt thời gian hết hạn hợp lý cho cache để làm mới dữ liệu định kỳ.
- **Giám sát liên tục:** Đánh giá định kỳ tỷ lệ Hit/Miss và feedback từ người dùng để căn chỉnh ngưỡng.

Distributed Hybrid Search: Cân bằng Sparse & Dense

VINUNIVERSITY

1. Bài toán trọng số cố định

- Cấu hình cố định (ví dụ: 50% BM25 + 50% Vector) không mang lại hiệu quả tối ưu cho mọi loại câu hỏi.
- Cần cơ chế điều chỉnh linh hoạt theo ngữ cảnh và ý định truy vấn của người dùng.

2. Giải pháp - Dynamic Weighting

- Sử dụng **Intent Classifier** để tự động phân loại ý định người dùng trước khi thực hiện tìm kiếm.
- **Keyword-heavy:** Ưu tiên chính xác từ khóa đối với mã lỗi, tên sản phẩm hoặc thuật ngữ cố định.
- **Conceptual:** Tập trung vào ngữ nghĩa đối với các câu hỏi mang tính khái niệm, tư duy và giải thích.

Loại câu hỏi (Query Type)	Tỷ trọng BM25 (Sparse)	Tỷ trọng Vector (Dense)
Keyword-heavy Mã lỗi, tên sản phẩm	**80%**	20%
Conceptual Câu hỏi khái niệm, tư duy	20%	**80%**

💡 *Trọng số linh hoạt giúp tối ưu đồng thời cả Precision (cho từ khóa) và Recall (cho ngữ nghĩa).*

Tăng tốc Reranking: Vượt qua Nút cổ chai PyTorch

VINUNIVERSITY

Nút cổ chai PyTorch

- **Độ trễ cao:** Cross-Encoder cung cấp độ chính xác vượt trội nhưng gặp điểm nghẽn hiệu năng trên nền tảng PyTorch tiêu chuẩn.
- **Ảnh hưởng hệ thống:** Tăng thời gian phản hồi nghiêm trọng khi xử lý danh sách top-50 ứng viên trong pipeline reranking.

Chuyển đổi Engine suy luận

- **ONNX Runtime / TensorRT:** Chuyển đổi mô hình sang các runtime được tối ưu hóa cho phần cứng.
- **Tối ưu Graph:** Loại bỏ các thao tác thừa và tối ưu hóa bộ nhớ đệm (Kernel Fusion).
- **Tăng Tốc Độ:** Giảm đáng kể độ trễ thực thi (Latency) so với PyTorch gốc.

Lượng tử hóa INT8

- **Quantization (INT8):** Giảm kích thước model và dấu chân bộ nhớ (Memory Footprint).
- **Bảo toàn chất lượng:** Duy trì độ chính xác xếp hạng (NDCG Score) gần như không đổi.
- **Tối ưu chi phí:** Đạt throughput cao hơn trên cùng hạ tầng phần cứng.

Kết quả kỳ vọng Production

- **Tối ưu toàn diện:** Tăng tốc độ phục vụ lên gấp nhiều lần, đáp ứng yêu cầu độ trễ thấp (Low-latency) cho hệ thống Tìm kiếm & Gợi ý.
- **Khả năng mở rộng:** Cho phép mở rộng quy mô reranking cho số lượng ứng viên lớn hơn mà không làm tăng chi phí phần cứng.

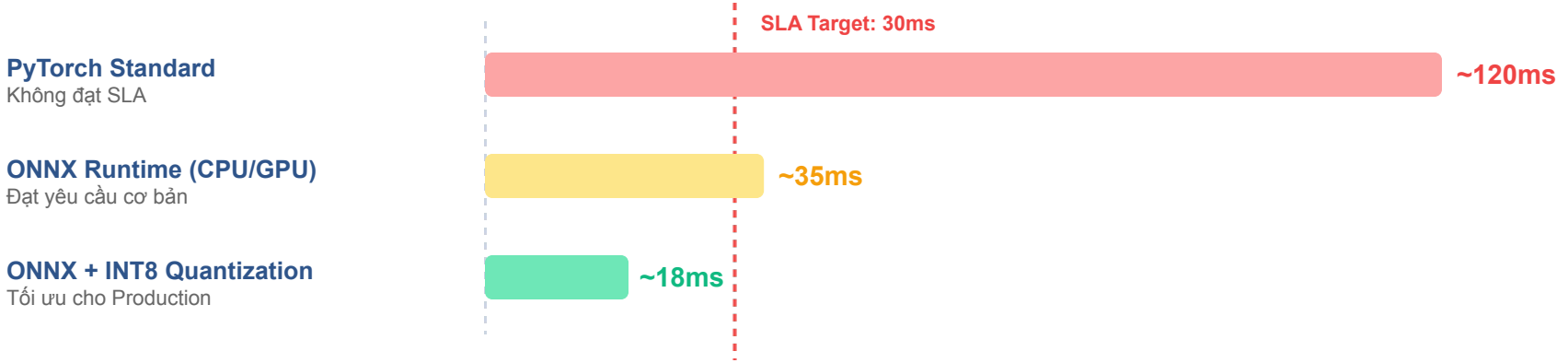
Benchmark: Độ trễ Rerank dưới 30ms

VINUNIVERSITY

Mục tiêu Performance SLA

• Đưa thời gian chạy rerank cho **top-50 candidates** xuống dưới **30ms** để đáp ứng điều kiện Production.

So sánh thời gian xử lý (Latency - ms) cho Top-50 Candidate



💡 *Kết hợp ONNX Runtime và Lượng tử hóa INT8 giúp giảm **~85% độ trễ**, vượt mục tiêu SLA 30ms.*

Vector DB Decision Framework cho Production

VINUNIVERSITY

Quyết định lựa chọn hạ tầng Vector Database phụ thuộc hoàn toàn vào quy mô dữ liệu, kiến trúc hệ thống hiện tại và yêu cầu vận hành thực tế của doanh nghiệp.

Qdrant

High-Throughput Filtering

- **Điểm mạnh cốt lõi:** Tối ưu cho việc lọc metadata tốc độ cao.
- **Ưu điểm:** Hiệu năng search & filter kết hợp cực nhanh nhờ Payload Indexing mạnh mẽ.
- **Phù hợp:** Hệ thống RAG cần điều kiện lọc thuộc tính phức tạp thời gian thực.

Milvus

Massive Scale

- **Điểm mạnh cốt lõi:** Phù hợp cho hệ thống quy mô lớn (hàng chục/trăm triệu vector).
- **Ưu điểm:** Kiến trúc Distributed Cloud-Native, mở rộng linh hoạt theo chiều ngang.
- **Phù hợp:** Enterprise Big Data, hệ thống tìm kiếm hình ảnh & đề xuất quy mô khổng lồ.

Pinecone

Managed Simplicity

- **Điểm mạnh cốt lõi:** Lựa chọn hàng đầu khi cần quản lý đơn giản.
- **Ưu điểm:** Fully Managed SaaS, không tốn chi phí vận hành hạ tầng, triển khai tức thì.
- **Phù hợp:** AI Startups, đội ngũ muốn tập trung làm sản phẩm thay vì Ops.

pgvector

Transactional Consistency

- **Điểm mạnh cốt lõi:** Tối ưu cho stack PostgreSQL sẵn có, đảm bảo tính nhất quán giao dịch.
- **Ưu điểm:** Tận dụng hạ tầng SQL hiện có, đảm bảo chuẩn ACID và tính toàn vẹn dữ liệu.
- **Phù hợp:** Ứng dụng đã dùng Postgres, quy mô dữ liệu vừa & nhỏ.

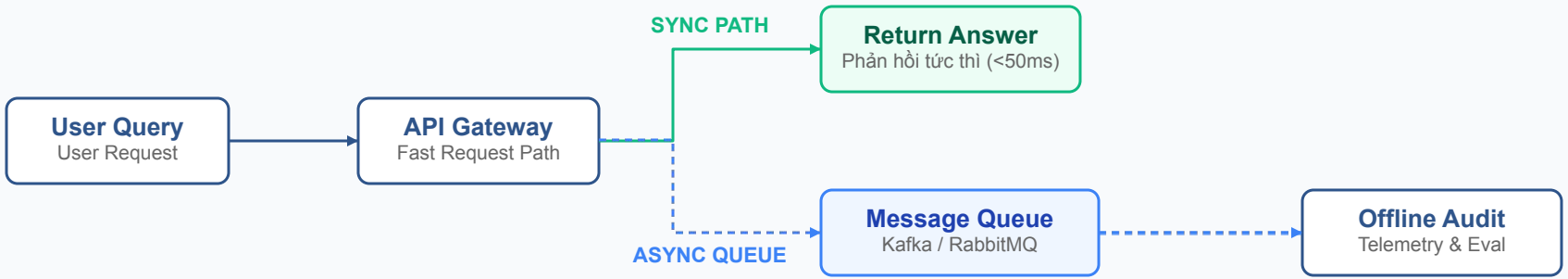
PHẦN 08

Production Observability, Drift & CI/CD

8

Vận hành và giám sát hệ thống RAG bền vững: Thu thập telemetry bất đồng bộ, kiểm soát độ lệch dữ liệu (drift), tự động hóa kiểm định hồi quy qua CI/CD và thử nghiệm an toàn với A/B Testing.

Continuous Monitoring: Telemetry Bất đồng bộ



1. Vấn đề: Sync Logging

Ghi log trực tiếp trong luồng request làm tăng độ trễ và dễ gây sập API khi hệ thống quá tải.

2. Giải pháp: Asynchronous Logging

Đẩy các cặp dữ liệu **[Query - Context - Answer]** vào hàng đợi nền (Message Queue / Kafka / RabbitMQ).

3. Lợi ích: Tối ưu Độ trễ & Khả năng Kiểm toán

- Thực hiện kiểm toán (audit) và đánh giá chất lượng ngoại tuyến hoàn toàn độc lập.
- **Không ảnh hưởng** đến độ trễ phản hồi (latency) trả về cho người dùng cuối.

Tracking Retrieval Drift: Khi Dữ liệu & Query "Lệch Pha"

VINUNIVERSITY

1. Hiện tượng (Concept Drift)

- Ngôn ngữ và xu hướng câu hỏi của người dùng **thay đổi liên tục** theo thời gian.
- Từ vựng query phát sinh trôi dạt (drift), **mất đồng bộ** với tập tài liệu (corpus) đã index ban đầu.

2. Phương pháp giám sát (Vector Distance Monitoring)

- Đo lường **phân phối khoảng cách vector** từ query thực tế đến historical index.
- Theo dõi sự thay đổi về cosine distance / L2 distance theo chuỗi thời gian.
- Phát hiện sớm vùng không gian vector bị suy giảm độ tương đồng (retrieval degradation).

3. Hành động kích hoạt (Automated Triggers)

- **Tự động cảnh báo (Alerting):** Kích hoạt khi chỉ số độ lệch khoảng cách vượt ngưỡng cho phép.
- **Cập nhật Corpus:** Bổ sung thêm dữ liệu mới chứa thuật ngữ/xu hướng mới.
- **Re-index & Fine-tune:** Tiến hành fine-tune embedding model để tái định vị không gian biểu diễn.

Automated Regression Testing: Bộ dữ liệu chuẩn (Golden Dataset)

VINUNIVERSITY

1. Nguồn gốc Golden Dataset

- Khai thác và tinh chỉnh hơn **200+ câu hỏi chuẩn** từ log thực tế của production.
- Chuẩn hóa kèm theo câu trả lời mẫu (Ground Truth) để làm thước đo cố định.

2. Tích hợp CI/CD Pipeline

- **Tự động kích hoạt:** Chạy toàn bộ bộ test mỗi khi có pull request hoặc thay đổi mới.
- **Phạm vi giám sát:** Kiểm soát biến động từ thay đổi code, prompt engineering, đến cấu hình chunking.
- Đảm bảo tính liên tục và nhất quán của chất lượng mô hình theo thời gian.

3. Cơ chế Gatekeeper (Cổng Gác Tự Động)

- **Chặn deploy tự động:** Ngăn chặn triển khai lên production nếu các chỉ số không đạt ngưỡng (threshold).
- **Bảo vệ hệ thống:** Loại bỏ rủi ro suy giảm độ chính xác (regression) trước khi ảnh hưởng người dùng.

CI/CD Quality Gates: Ngưỡng Chấp nhận Production

Tiêu chí Đánh giá (Evaluation Metric)	Ngưỡng Chuẩn	Mục tiêu Đảm bảo	Trạng thái CI/CD
Context Recall Độ bao phủ thông tin nền	≥ 0.80	Không bỏ sót thông tin nền quan trọng	PASSED / ALLOW
Faithfulness Độ trung thực của phản hồi	≥ 0.80	LLM không bịa đặt, bám sát context	PASSED / ALLOW

Quy tắc Vận hành Quality Gate

- Đánh giá tự động được tích hợp trực tiếp vào **Pipeline CI/CD** trước khi deployment.
- Mọi bản cập nhật (Prompt, Model, RAG Index) bắt buộc phải vượt qua cả 2 ngưỡng kiểm định.
- Đảm bảo hệ thống vận hành an toàn và đáp ứng tiêu chuẩn chất lượng production.

Cơ chế Chặn Lỗi Tự động (Build Failure)

- **Trạng thái kích hoạt:** Nếu điểm số **Context Recall** hoặc **Faithfulness < 0.80**.
- **Hành động tự động:** Hệ thống lập tức kích hoạt trạng thái **BUILD FAILED**.
- **Kết quả:** Ngăn chặn hoàn toàn việc phát tán mô hình hoặc dữ liệu lỗi ra môi trường thật (Production).

Thử nghiệm An toàn: Shadow Deployments & A/B Testing

VINUNIVERSITY

Shadow Deployment (Chạy ngầm đo đạc)

- Sao chép một phần traffic thực tế sang cấu hình retrieval thử nghiệm ngầm.
- Đo lường độ trễ và chất lượng phản hồi mà hoàn toàn không ảnh hưởng tới người dùng.

A/B Testing (Chia tải thực tế)

- Chia tải trực tiếp cho người dùng (ví dụ: 90% traffic cho Production cũ, 10% cho cấu hình mới).

Mục tiêu Đo lường

- Đánh giá chính xác mức độ tương tác của người dùng (user engagement) và tác động độ trễ trước khi roll out toàn bộ.

Hands-on 18

Đủ lý thuyết — giờ tự tay dựng pipeline trên dữ liệu thật, đo bằng RAGAS, và phân tích thất bại như TaiLieuBot

1

Cá nhân — 2h implement + 30 phút reflection

Implement **tất cả 5 module** (TODO markers trong code):

1. `m1_chunking.py` — semantic, hierarchical, structure-aware
2. `m2_search.py` — BM25 Vietnamese + Dense + RRF
3. `m3_rerank.py` — CrossEncoder rerank top-20 → top-3
4. `m4_eval.py` — RAGAS 4 metrics + failure analysis
5. `m5_enrichment.py` — contextual / summary / hypo-Q&A

Verify: `pytest tests/test_m*.py` · Pipeline: `python src/pipeline.py`

Reflection (30 phút): map lecture concept → code vừa viết · khó khăn & cách giải · plan cho project cá nhân

`git clone .../lab18` → `docker compose up -d` (Qdrant) → `pip install -r requirements.txt` → `python naive_baseline.py` (lấy điểm gốc để so).

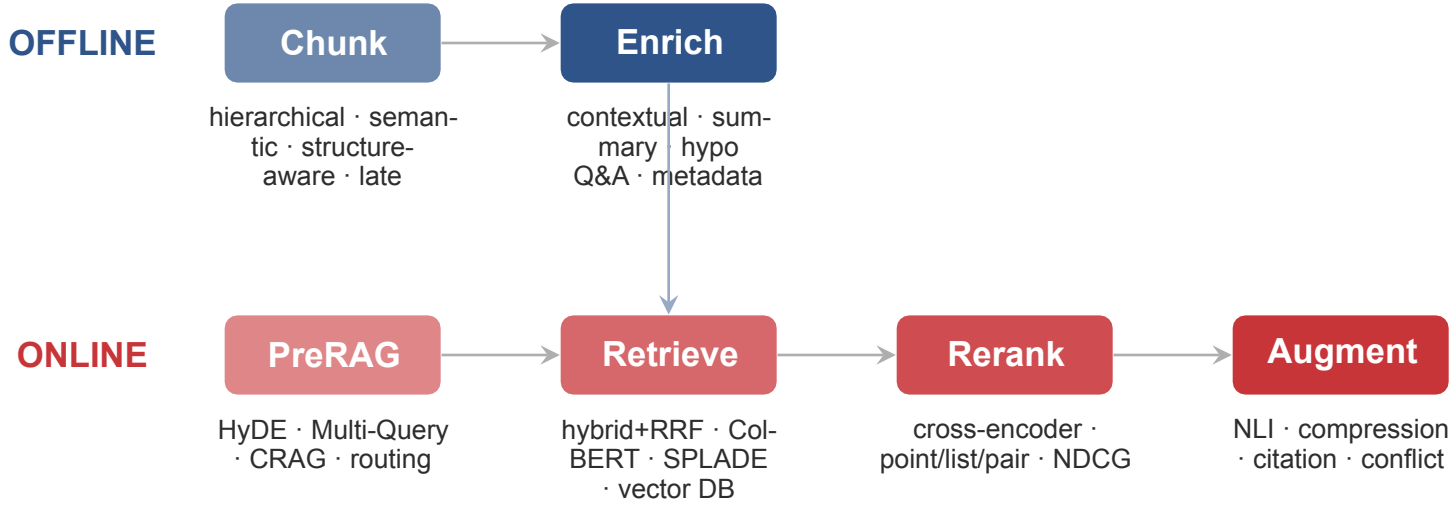
So Sánh: Naive vs Production vs +Nâng Cao

Pipeline	Cấu hình	Kỳ vọng RAGAS (minh hoạ)
A — Naive	paragraph chunk 512 + dense-only	Faithfulness ~0.70, Recall ~0.55
B — Production	hierarchical + hybrid + bge-reranker	Faithfulness ≥0.85, Recall ≥0.75
C — Tự chọn	B + **1 hoặc nhiều** kỹ thuật nâng cao (tự chọn)	đo delta vs B

Thêm vào B **một hoặc nhiều** kỹ thuật rồi đo cải thiện. Gợi ý: contextual embeddings · summary/hypothetical-Q&A enrichmetadata filter · HyDE · Multi-Query · semantic chunk · threshold rerank. Không giới hạn ở contextual.

Lưu ý: Cùng dữ liệu, cùng judge model — chênh lệch điểm đến **phần lớn** từ kỹ thuật pipeline. Con số minh hoạ; số thật đo trong lab.

Mỗi Tầng Có Núm Vặn Riêng — Vặn Đúng Tầng, Đừng Đổi Model



RAG yếu trong production → (1) đo RAGAS biết **retrieval** hay **generation** hỏng → (2) Error Tree map vấn đề ra **tầng** → (3) bản đồ này chọn **núm vặn**. Đừng đổi model, vặn đúng tầng. (Đo→Day24 · embedding mù→GraphRAG Day19 / HITL Day27.)

Key Takeaways

Production RAG là kỹ thuật chẩn đoán + sửa đúng tầng, không phải thêm model mạnh hơn

2

Tổng kết — Key Takeaways

1. **Chẩn đoán trước, sửa sau.** RAG = OFFLINE + ONLINE; dùng Error Tree để biết tầng nào mất điểm.

2. **OFFLINE đặt trần chất lượng.** Hierarchical chunk + contextual + enrichment: làm 1 lần lúc index, lợi mọi query — retrieval không thể tốt hơn cái đã index.

3. **Reranking ROI cao khi recall đã đủ.** +15–25% precision với ~30–50ms *khi latency cho phép*; vô dụng nếu chunk đúng chưa được retrieve. Chọn theo ngôn ngữ & chi phí.

4. **Đo để sửa đúng chỗ.** RAGAS 4 metric → diagnostic tree → sửa tầng yếu (eval sâu: Day 24).

5. **Biết giới hạn.** Embedding mù negation/temporal — chẩn đoán loại lỗi trước khi thêm data.

Tài Liệu Tham Khảo

1. Anthropic. *Introducing Contextual Retrieval* (2024).
   `https://www.anthropic.com/news/contextual-retrieval`
2. Weaviate. *Advanced RAG T echniques* ebook.
3. ES Khattab & Zaharia. *ColBERT: Late Interaction*. SIGIR 2020.
4. Sarthi et al. *RAPTOR: Recursive Abstractive Tree*. ICLR 2024.
5. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS 2022.
6. Faysse et al. *ColPali: Vision-Language Retrieval*. arXiv:2407.01449 (2024).
7. Gao et al. *Precise Zero-Shot Dense Retrieval (HyDE)*. arXiv:2212.10496
8. Asai et al. *Self-RAG*. arXiv:2310.11511 (đọc thêm → Day 23)
9. Yan et al. *Corrective RAG (CRAG)*. arXiv:2401.15884
10. Es et al. *RAGAS*. arXiv:2309.15217
11. Ehtesham et al. *Agentic RAG Survey*. arXiv:2501.09136 (đọc thêm → Day 20/23)

Tiếp theo & Bài tập

Day 19 — GraphRAG & Knowledge Graphs

"Khi user hỏi về quan hệ giữa 5 entity, flat RAG trả lời sai — GraphRAG trả lời đúng."

- Hoàn thiện Lab 18: pipeline + RAGAS report đạt ≥0.75
- Thử 1 cải tiến M5 khác, đo delta vs baseline
- Đọc Microsoft GraphRAG paper (2024) trước Day 19

Hỏi & Đáp

Production RAG: bạn sẽ sửa tầng nào đầu tiên trong hệ thống của mình?



Cảm ơn!

Trần Quang Thiện · AICB Phase 2 · Track 3

Production RAG — Ngày 18